Vom Projektordner zum GitHub-Repository

Ein Leitfaden für Einsteiger: Schritt für Schritt mit Git Bash unter Windows, von der Planung bis zur Veröffentlichung

Was du am Ende kannst: einen lokalen Projektordner sauber vorbereiten, daraus ein Git-Repository machen, Änderungen nachvollziehbar festschreiben, das Projekt auf GitHub hochladen, dort weiterpflegen und einen fertigen Stand als Release veröffentlichen, ohne dabei Zugangsdaten oder deine private Mailadresse preiszugeben.

Voraussetzungen: ein Windows-Rechner, ein GitHub-Konto und etwas Neugier. Vorkenntnisse in Git brauchst du nicht.

Version 1.1 · November 2026

Inhalt

1 So liest du diesen Leitfaden 3
2 Die wichtigsten Begriffe 3
3 Einmalig: GitHub-Konto vorbereiten 4
4 Einmalig: Git installieren und einrichten 4
5 Das Projekt planen 6
6 Das lokale Repository anlegen 10
7 Im Alltag: Änderungen festschreiben 12
8 Das Repo auf GitHub anlegen und hochladen 13
9 Weiterarbeiten: lokal und im Web 14
10 Einen fertigen Stand als Release veröffentlichen 15
11 Wenn etwas klemmt 15
12 Befehlsübersicht 17
13 Checkliste 18

1 So liest du diesen Leitfaden

Der Leitfaden ist so aufgebaut, dass jedes Kapitel auf dem vorigen aufbaut. Die ersten Kapitel erledigst du nur einmal pro Rechner, ab Kapitel 5 folgt der Ablauf, den du für jedes neue Projekt wiederholst. Kapitel 10 und 11 sind Nachschlagewerk für den Fall, dass etwas klemmt.

Platzhalter

Alles in spitzen Klammern ersetzt du durch deine eigenen Angaben, und zwar **ohne** die Klammern:

Platzhalter	Bedeutung	Beispiel
`<GITHUB-NAME>`	Dein Benutzername auf GitHub	`tomlahr`
`<ID>`	Die Zahl vor deiner GitHub-Ersatzadresse (Kapitel 3)	`12345678`
`<PROJEKT>`	Name des Projektordners und späteren Repos, ohne Leerzeichen	`Pico-Wetterstation`
`<DATEI>`	Name einer Datei im Projekt	`README.md`
`<NACHRICHT>`	Kurze Beschreibung einer Änderung	`README um Verdrahtung ergänzt`

Befehle eingeben

Alle grau hinterlegten Zeilen gibst du in der **Git Bash** ein und bestätigst jede mit Enter. Zeilen, die mit # beginnen, sind Kommentare zur Erklärung; die musst du nicht eintippen.

```
# Das ist ein Kommentar
git status
```

Tipp: In der Git Bash funktioniert Strg+V oft nicht. Einfügen geht mit **Umschalt+Einfg** oder **Rechtsklick → Paste**. Zum Kopieren markierst du Text einfach mit der Maus.

Achtung: Mehrere lesende Befehle (`status`, `log`, `diff`) darfst du gemeinsam einfügen. Befehle, die etwas verändern oder löschen, gibst du einzeln ein und liest die Ausgabe, bevor der nächste folgt.

2 Die wichtigsten Begriffe

Git und GitHub sind zwei verschiedene Dinge, die zusammenarbeiten:

- **Git** ist ein Programm auf deinem Rechner. Es merkt sich jede Version deiner Dateien.
- **GitHub** ist eine Website, auf der du diese Versionsgeschichte online ablegst, sicherst und anderen zeigen kannst.

Begriff	Bedeutung im Alltag
Repository (Repo)	Ein Projektordner, dessen Geschichte Git verwaltet. Die Geschichte liegt im versteckten Unterordner `.git`.

Begriff	Bedeutung im Alltag
Commit	Ein festgeschriebener Zwischenstand mit Nachricht, Autor und Datum. Wie ein Foto deines Projekts zu einem Zeitpunkt.
Staging (`git add`)	Das Vormerken von Änderungen für den nächsten Commit. Du entscheidest, was in das nächste „Foto“ kommt.
Branch	Eine Entwicklungslinie. Für den Einstieg genügt der Hauptzweig `main`.
Remote / `origin`	Die Verbindung zum Repo auf GitHub. `origin` ist ihr üblicher Name.
Push	Lokale Commits zu GitHub hochladen.
Fetch / Pull	Stand von GitHub holen. Fetch schaut nur nach, Pull übernimmt die Änderungen auch.
Tag / Release	Eine dauerhafte Markierung eines Commits als Version, z. B. `v1.0`.

Der Weg einer Änderung sieht immer gleich aus:

```
Datei ändern  →  git add  →  git commit  →  git push  →  GitHub
 (Arbeitsordner)  (vormerken)  (festschreiben)  (hochladen)
```

3 Einmalig: GitHub-Konto vorbereiten

Git schreibt in jeden Commit eine Mailadresse, und die ist in öffentlichen Repos für jeden lesbar. Programme suchen GitHub gezielt nach solchen Adressen ab. Deshalb nutzt du eine Ersatzadresse, die GitHub dir kostenlos bereitstellt.

Schritt 3.1: Ersatzadresse aktivieren

- Auf GitHub oben rechts auf dein Profilbild → **Settings**.
- Links **Emails** wählen.
- Häkchen bei **Keep my email addresses private** setzen.
- Die angezeigte Adresse der Form `<ID>+<GITHUB-NAME>@users.noreply.github.com` notieren.
- Häkchen bei **Block command line pushes that expose my email** setzen. Dann lehnt GitHub jeden Upload ab, in dem versehentlich deine echte Adresse steckt.

> **Tipp:** Die Ersatzadresse ist dafür gedacht, öffentlich zu sein. Du musst sie nicht geheim halten.

4 Einmalig: Git installieren und einrichten

Schritt 4.1: Git für Windows installieren

Lade „Git for Windows“ von `git-scm.com` herunter und installiere es. Die vorgeschlagenen Einstellungen im Installationsassistenten kannst du übernehmen. Danach findest du im Startmenü die **Git Bash**, ein Konsolenfenster ähnlich wie unter Linux.

```
# In der Git Bash: Läuft Git?
git --version
```

Schritt 4.2: Identität festlegen

Diese Angaben gelten für alle Repos auf diesem Rechner (`--global`). Du machst das nur einmal.

```
git config --global user.name "<GITHUB-NAME>"
git config --global user.email "<ID>+<GITHUB-NAME>@users.noreply.github.com"
```

> **Achtung:** Trage hier **niemals** deine echte Mailadresse ein. Sie stünde sonst dauerhaft in jedem Commit, auch nach späterem Löschen der Dateien.

Schritt 4.3: Weitere Grundeinstellungen

```
# Hauptzweig heißt "main" wie auf GitHub
git config --global init.defaultBranch main

# Windows- und Linux-Zeilenenden automatisch angleichen
git config --global core.autocrlf true

# Beim Abholen von GitHub zusammenführen statt nachfragen
git config --global pull.rebase false

# Kontrolle: alles anzeigen
git config --global --list
```

In der Liste sollten Name, Ersatzadresse, `main`, `true` und `false` auftauchen. Steht dort eine echte Mailadresse, wiederholst du einfach den Befehl aus Schritt 4.2.

Schritt 4.4: Arbeitsordner anlegen

Lege einen festen Ordner für alle Repos an, zum Beispiel `C:\GitHub`. Er sollte **nicht** in einem Cloud-Ordner wie OneDrive liegen, weil sich Synchronisierung und Git gegenseitig stören können.

> **Achtung:** Ist an deinem Rechner der Windows-Administratorschutz aktiv, prüfe, wem der Ordner gehört: Rechtsklick → Eigenschaften → Sicherheit → Erweitert → **Besitzer**. Dort sollte dein eigenes Benutzerkonto stehen. Sonst meldet Git später „dubious ownership“ (siehe Kapitel 11).

Schritt 4.5: In der Git Bash zurechtfinden

Aufgabe	So geht's
Pfad angeben	Windows `C:\GitHub` heißt in der Bash `/c/GitHub`
Ordner wechseln	`cd /c/GitHub/<PROJEKT>`
Ordnerinhalt anzeigen	`ls` (versteckte Dateien mit `ls -a`)
Namen ergänzen	Anfang tippen, dann **Tab-Taste**
Leerzeichen im Namen	Pfad in Anführungszeichen: `cd "/c/GitHub/Mein Ordner"`
Bash direkt im Ordner öffnen	Explorer: Rechtsklick auf den Ordner → Weitere Optionen anzeigen → **Open Git Bash here**

5 Das Projekt planen

Ein gut geplanter Ordner erspart dir später Aufräumarbeit. Überlege vor dem ersten Commit, was hinein gehört und was nicht.

Schritt 5.1: Namen festlegen

- Keine Leerzeichen, keine Umlaute. Bindestriche trennen Wörter: `Pico-Wetterstation`.
- Ordner und späteres Repo heißen gleich. GitHub erlaubt ohnehin keine Leerzeichen in Repo-Namen.

Schritt 5.2: Die Projektart bestimmen

Nicht jedes Repo enthält Programmcode. GitHub eignet sich für alles, was aus Dateien besteht und sich im Lauf der Zeit verändert: Texte, Anleitungen, Daten, Einstellungen. Je nach Art des Inhalts bewährt sich eine andere Ordnerstruktur. Bestimme deshalb zuerst, welche Art von Projekt du anlegst.

Projektart	Beispiele	Typische Lizenz	Sichtbarkeit
A Längeres Schriftwerk	Buch, Abhandlung, Handbuch	CC BY 4.0	Öffentlich oder privat
B Kurzes Dokument	Artikel, Précis, Bedienungs- oder Einrichtungsanleitung	CC BY 4.0	Meist öffentlich
C DIY-Projekt mit Code	Pico, ESP32, Raspberry Pi	MIT	Meist öffentlich
D Daten-Repo	Messreihen, Verbrauchswerte	CC BY 4.0 oder keine	Meist privat
E Konfigurationssammlung	Home Assistant, ESPHome, Skripte	MIT oder keine	Fast immer privat
F Wissenssammlung	Notizen, Spickzettel, Linklisten	CC BY 4.0 oder keine	Öffentlich oder privat

Zwei Begriffe aus der Tabelle kurz erklärt: **MIT** ist eine Lizenz für Software. Sie erlaubt jedem, den Code zu nutzen und zu verändern, solange dein Name erhalten bleibt. **CC BY 4.0** (Creative Commons, Namensnennung) leistet dasselbe für Texte, Bilder und Daten. Für Texte passt sie besser als eine Software-Lizenz. „Keine“ heißt: Bei privaten Repos, die niemand außer dir sieht, brauchst du keine Lizenz.

> **Tipp:** Ein paar Regeln gelten für alle Arten: Datei- und Ordnernamen klein, ohne Leerzeichen und Umlaute. Wo die Reihenfolge zählt, stellst du Nummern voran: `01-einleitung.md`, `02-grundlagen.md`. Und Texte schreibst du in **Markdown** (`.md`) statt in Word: Git zeigt dann jede Änderung zeilengenau, bei Word-Dateien sieht es nur „Datei geändert“.

A Längeres Schriftwerk: Buch, Abhandlung, Handbuch

Jedes Kapitel bekommt eine eigene Datei. So siehst du in der Versionsgeschichte sofort, in welchem Kapitel du was geändert hast, und behältst auch bei hundert Seiten den Überblick.

```
<BUCHTITEL>/
├── README.md              # Worum geht es? Inhaltsverzeichnis mit Links
├── LICENSE                # z. B. CC BY 4.0
├── .gitattributes         # Bilder und PDFs als binär kennzeichnen
├── kapitel/
│   ├── 00-vorwort.md
│   ├── 01-einleitung.md
│   ├── 02-grundlagen.md
│   └── 03-praxis.md
├── bilder/
│   ├── abb-01-uebersicht.png
│   └── abb-02-aufbau.jpg
└── quellen/
    └── literatur.md       # Quellenangaben, Links
```

- Die README dient als Inhaltsverzeichnis. Links auf Kapitel schreibst du relativ: `[Einleitung](kapitel/01-einleitung.md)`. GitHub macht daraus klickbare Verweise.
- Bilder bindest du im Kapitel ein mit `![Beschreibung](../bilder/abb-01-uebersicht.png)`. Die zwei Punkte bedeuten „eine Ordnerebene höher“.
- Die fertige PDF-Fassung legst du am besten **nicht** ins Repo, sondern hängst sie an ein Release (Kapitel 10). Git speichert eine PDF bei jeder Änderung komplett neu, das bläht das Repo auf.

B Kurzes Dokument: Artikel, Précis, Anleitung

Für Texte, die auf wenige Seiten passen, reicht eine flache Struktur ohne Unterordner für Kapitel.

```
<ANLEITUNG>/
├── README.md              # Kurzbeschreibung, Version, Link zum Dokument
├── LICENSE                # z. B. CC BY 4.0
├── .gitattributes         # *.pdf binary, *.png binary
├── anleitung.md           # der eigentliche Text
├── befehle.txt            # optional: Befehle zum Kopieren
└── bilder/
    ├── schritt-01.png
    └── schritt-02.png
```

- Bei einem Précis oder Artikel genügt oft sogar nur `README.md` plus `artikel.md`.
- Die Versionsnummer gehört an eine feste Stelle, etwa oben in die README. Fertige Stände veröffentlichst du als Release.
- Bildschirmfotos vor dem Hochladen prüfen: Benutzernamen, Gerätenamen, Netzwerknamen oder IP-Adressen lassen sich leicht übersehen.

> **Achtung:** Eine PDF-Fassung im Repo ist bei kurzen Dokumenten vertretbar. Dann aber unbedingt `*.pdf binary` in die `.gitattributes` eintragen, sonst kann Git die Datei beschädigen (Schritt 6.3).

C DIY-Projekt mit Code: Mikrocontroller, Raspberry Pi

Hier trennt der Ordner `src` sauber, was auf das Gerät kommt, von dem, was nur der Dokumentation dient. Wer dein Projekt nachbauen will, kopiert einfach den Inhalt von `src`.

```
<PROJEKT>/
├── README.md              # Hardware, Verdrahtung, Installation, Bedienung
├── LICENSE                # z. B. MIT
├── .gitignore             # secrets.py, vom Gerät erzeugte Dateien
├── .gitattributes
├── src/                   # alles, was aufs Gerät kommt
│   ├── main.py
│   ├── config.py          # Einstellungen, Pins, Kalibrierung
│   ├── secrets_example.py  # Vorlage mit Platzhaltern
│   └── lib/               # nur falls auch auf dem Gerät vorhanden
├── docs/
│   ├── verdrahtung.png
│   └── aufbau-foto.jpg
└── hardware/              # optional
    ├── schaltplan.pdf
    └── gehaeuse.stl       # 3D-Druck-Vorlage
```

- Die Struktur in `src` entspricht genau dem Stammverzeichnis auf dem Gerät.
- Dateien, die das Gerät im Betrieb selbst anlegt (Messwert-Speicher, letzte Einstellungen), gehören in die `.gitignore`. Sie beschreiben den Zustand deines Geräts, nicht den Code.
- Arbeitest du mit Werkzeugen wie PlatformIO oder der Arduino-IDE, geben diese eine eigene Struktur vor (etwa `platformio.ini`, `src/`, `include/`, `lib/`). Folge dann deren Vorgabe, statt eine eigene zu erfinden.
- Datenblätter und fremde Bibliotheken verlinkst du in der README, statt sie hochzuladen.

D Daten-Repo: Messreihen, Verbrauchswerte

Git eignet sich gut, um Datenreihen zu pflegen, weil jede neue oder korrigierte Zeile in der Versionsgeschichte sichtbar wird. Wichtigster Teil ist dabei die Beschreibung, woher die Zahlen stammen.

```
<DATENREIHE>/
├── README.md              # Herkunft, Methodik, Lücken, Einheiten
├── .gitattributes         # *.png binary, *.xlsx binary
├── daten/
│   ├── 2025.csv
│   └── 2026.csv
└── auswertung/
    ├── auswertung.py      # optional: Skript, das die Grafiken erzeugt
    ├── verlauf-2025.png
    └── verlauf-2026.png
```

- **CSV statt Excel** als eigentliche Datenquelle: CSV ist reiner Text, Git zeigt jede geänderte Zeile. Eine Excel-Datei mit Formeln darf zusätzlich existieren, ist aber nicht die „Wahrheit“.
- In der README festhalten: Woher kommen die Werte? Welcher Zeitraum? Welche Einheit? Wo gibt es Lücken oder Schätzungen? Ohne diese Angaben weißt du ein Jahr später nicht mehr, was die Zahlen bedeuten.

- Grafiken (PNG) sind Momentaufnahmen. Kommen neue Daten hinzu, musst du sie neu erzeugen.
- GitHub warnt bei Dateien über 50 MB und lehnt Dateien über 100 MB ab. Große Datenmengen teilst du deshalb in Jahres- oder Monatsdateien auf.

> **Achtung:** Nutzungsdaten verraten indirekt viel über dein Leben, etwa wann du zu Hause bist oder verreist. Solche Repos stellst du auf **Private**.

E Konfigurationssammlung: Home Assistant, ESPHome, Skripte

Wer seine Einstellungen in Git ablegt, kann jede Änderung nachvollziehen und nach einem misslungenen Experiment auf den letzten funktionierenden Stand zurückgehen.

```
<KONFIGURATION>/
├── README.md              # Welches Gerät, welche Version, was steckt wo
├── .gitignore             # secrets.yaml, Datenbanken, Logs
├── homeassistant/
│   ├── configuration.yaml
│   ├── automations.yaml
│   ├── scripts.yaml
│   ├── secrets_example.yaml   # Vorlage mit Platzhaltern
│   └── packages/
├── esphome/
│   ├── wohnzimmer-sensor.yaml
│   └── secrets_example.yaml
└── skripte/
    └── backup.sh
```

- Zugangsdaten liegen bei Home Assistant und ESPHome in `secrets.yaml`. Diese Datei gehört immer in die `.gitignore`, ins Repo nur eine Vorlage mit Platzhaltern.
- Datenbanken und Protokolle (`*.db`, `*.log`) ebenfalls in die `.gitignore`: Sie ändern sich ständig und sind kein Bestandteil der Konfiguration.
- Gerätenamen, Raumbezeichnungen und IP-Adressen beschreiben deine Wohnung. Deshalb solche Repos **immer privat** halten, auch wenn keine Passwörter darin stehen.

F Wissenssammlung: Notizen, Spickzettel, Linklisten

Die einfachste Art: einzelne Markdown-Dateien zu einem Thema, die mit der Zeit wachsen. GitHub zeigt jede Datei formatiert an, mit Überschriften, Tabellen und klickbaren Links.

```
<THEMA>/
├── README.md              # Übersicht mit Links zu allen Seiten
├── grundlagen.md
├── befehle.md
├── links.md
└── bilder/
    └── beispiel.png
```

- Querverweise zwischen Seiten funktionieren relativ: `[Befehle](befehle.md)`.
- Kleine, häufige Commits lohnen sich hier besonders. So siehst du später, wann du was gelernt oder korrigiert hast.
- Wächst die Sammlung über ein Dutzend Seiten, gliedere sie in Unterordner nach Teilthemen.

Schritt 5.3: Was gehört nicht in ein Repo?

Nicht hochladen	Stattdessen
Zugangsdaten (WLAN, Passwörter, Tokens)	Eigene Datei wie `secrets.py`, in `.gitignore` eintragen; ins Repo nur eine Vorlage `secrets_example.py` mit Platzhaltern
Private Angaben (Router, Zeiten, Wohnung)	Private Notizen außerhalb des Repos
ZIP-Archive mit Code	Entpackte Dateien. GitHub kann in ZIPs nicht hineinschauen.
Fremde Datenblätter und Handbücher	In der README verlinken
Automatisch erzeugte Dateien	In `.gitignore` eintragen

> **Achtung:** Was einmal committet und hochgeladen ist, bleibt in der Versionsgeschichte abrufbar, auch wenn du die Datei später löschst. Ein versehentlich veröffentlichtes Passwort musst du deshalb **sofort ändern**.

Schritt 5.4: README vorbereiten

Die `README.md` zeigt GitHub auf der Startseite des Repos an. Sie sollte mindestens beantworten: Was ist das Projekt? Was brauche ich dafür? Wie setze ich es ein? Ein einfaches Grundgerüst:

```
# <PROJEKT>

Kurze Beschreibung in ein bis zwei Sätzen.

## Hardware / Voraussetzungen

## Installation

## Bedienung

## Lizenz

MIT, siehe LICENSE
```

6 Das lokale Repository anlegen

Jetzt wird aus dem Ordner ein Git-Repository. Die Reihenfolge ist wichtig: Die Schutzdateien kommen **vor** dem ersten Commit.

Schritt 6.1: In den Ordner wechseln und Git starten

```
cd /c/GitHub/<PROJEKT>
git init
```

Git meldet „Initialized empty Git repository“. In der Eingabezeile erscheint jetzt `(main)`, das Zeichen, dass du dich in einem Repo befindest.

Schritt 6.2: .gitignore anlegen

Diese Datei listet alles, was Git grundsätzlich ignorieren soll. Passe die Liste an dein Projekt an:

```
cat > .gitignore << 'EOF'
# Zugangsdaten
secrets.py
*.env

# Automatisch erzeugt
__pycache__/
*.log

# Systemdateien
Thumbs.db
desktop.ini
.DS_Store
EOF
```

Der Befehl `cat > DATEI << 'EOF'` schreibt alle folgenden Zeilen bis zum Wort `EOF` in die Datei. Du kannst die Datei genauso gut mit einem Texteditor anlegen.

Schritt 6.3: .gitattributes anlegen

Wegen `core.autocrlf` passt Git Zeilenenden an. Bei Binärdateien wie PDFs oder Bildern würde das die Datei beschädigen. Diese Regeln verhindern das:

```
cat > .gitattributes << 'EOF'
* text=auto
*.pdf binary
*.png binary
*.jpg binary
*.zip binary
*.xlsx binary
EOF
```

Schritt 6.4: Lizenz anlegen

Ohne Lizenz darf rechtlich niemand deine Inhalte weiterverwenden, auch wenn sie öffentlich sind. Welche Lizenz passt, steht in der Tabelle in Schritt 5.2: MIT für Code, CC BY 4.0 für Texte und Daten. Am einfachsten legst du sie später auf GitHub an (Kapitel 8). Alternativ kopierst du den Lizenztext von `choosealicense.com` in eine Datei `LICENSE` und ergänzt Jahr und Namen.

Schritt 6.5: Kontrollieren, was Git sieht

```
git status
```

Unter „Untracked files“ listet Git alle Dateien, die es noch nicht verwaltet. Prüfe die Liste genau: Taucht dort `secrets.py` auf, stimmt etwas mit der `.gitignore` nicht. Dann erst korrigieren, nicht weitermachen.

Schritt 6.6: Der erste Commit

```
# Alle Dateien vormerken
git add .

# Vorgemerktes anzeigen (grün = wird festgeschrieben)
git status

# Festschreiben
git commit -m "Erster Stand: Projektstruktur, README, Schutzdateien"
```

Der Punkt in `git add .` steht für „alles im aktuellen Ordner“, abzüglich dessen, was `.gitignore` ausschließt.

Schritt 6.7: Den Commit prüfen

```
git log --format="%h  %an <%ae>  %ad  %s" --date=short
```

Die Zeile zeigt Kurzkennung, Name, **Mailadresse**, Datum und Nachricht. Steht dort deine Ersatzadresse, ist alles richtig.

7 Im Alltag: Änderungen festschreiben

Solange dein Projekt lokal reift, wiederholst du diesen kleinen Kreislauf, so oft du willst.

Schritt 7.1: Der Arbeitszyklus

```
# 1. Was hat sich geändert?
git status

# 2. Was genau? (q beendet die Anzeige)
git diff

# 3. Vormerken: einzelne Datei oder alles
git add <DATEI>
git add .

# 4. Festschreiben
git commit -m "<NACHRICHT>"
```

Schritt 7.2: Gute Commit-Nachrichten

Deine Versionsgeschichte ist nur so nützlich wie ihre Nachrichten. Schreib so, dass du in einem Jahr noch weißt, was du warum geändert hast. Deutsch ist völlig in Ordnung.

Gut	Nicht hilfreich
„TEMP_OFFSET auf -3.2 nach Abgleich mit Referenzsensor“	„Änderung“
„README um Verdrahtungstabelle ergänzt“	„Update“
„WLAN-Wiederverbindung nach 3 Fehlversuchen“	„Fix“

Schritt 7.3: Einen Fehler rückgängig machen

Situation	Befehl
Änderung an einer Datei verwerfen (noch nicht vorgemerkt)	`git restore <DATEI>`
Datei aus dem Vormerken nehmen	`git restore --staged <DATEI>`
Letzte Commit-Nachricht korrigieren (noch nicht hochgeladen)	`git commit --amend -m "<NACHRICHT>"`

> **Achtung:** `git restore` verwirft Änderungen endgültig. Im Zweifel vorher den Ordner kopieren.

8 Das Repo auf GitHub anlegen und hochladen

Ist das Projekt reif genug, verbindest du es mit GitHub.

Schritt 8.1: Leeres Repo auf GitHub erstellen

- Auf GitHub oben rechts **+** → **New repository**.
- **Repository name:** genau `<PROJEKT>`, wie dein lokaler Ordner.
- **Description:** ein Satz, was das Projekt ist.
- **Sichtbarkeit:** zunächst **Private** wählen. Umstellen auf Public geht jederzeit später.
- **Keine** Häkchen bei „Add a README file“, „.gitignore“ oder „license“ setzen.
- **Create repository**.

> **Achtung:** Das Repo muss **leer** sein. Legt GitHub eine README an, existieren online und lokal zwei verschiedene erste Commits, und der erste Upload wird abgelehnt.

GitHub zeigt dir danach eine Seite mit Befehlen. Du brauchst davon nur die Adresse in der Form `https://github.com/<GITHUB-NAME>/<PROJEKT>.git`.

Schritt 8.2: Lokales Repo mit GitHub verbinden

```
git remote add origin https://github.com/<GITHUB-NAME>/<PROJEKT>.git

# Kontrolle
git remote -v
```

Die Kontrolle zeigt zweimal die Adresse, einmal mit „fetch“ (holen) und einmal mit „push“ (hochladen).

Schritt 8.3: Erster Upload

```
git push -u origin main
```

Beim allerersten Upload öffnet sich ein Fenster des **Git Credential Managers**. Du meldest dich im Browser bei GitHub an und erlaubst den Zugriff. Danach merkt sich Windows die Anmeldung; Passwörter musst du nirgends von Hand eintragen. Das `-u` verknüpft deinen lokalen Zweig dauerhaft mit GitHub, künftig reicht ein einfaches `git push`.

Schritt 8.4: Ergebnis auf GitHub prüfen

- Lade die Repo-Seite neu. Deine Dateien und die README erscheinen.
- Fehlt `secrets.py`? Gut, genau so soll es sein.
- Rechts erscheint nach kurzer Zeit unter „Languages“ die erkannte Programmiersprache.

Schritt 8.5: Repo vervollständigen (Weboberfläche)

Aufgabe	So geht's
Lizenz	Add file → Create new file → Name `LICENSE` → Choose a license template → passende Lizenz wählen (Schritt 5.2) → Jahr und Name eintragen → Commit changes. Fehlt CC BY 4.0 in der Auswahl, legst du die Datei mit dem Text von `choosealicense.com/licenses/cc-by-4.0` an.
Beschreibung und Topics	Zahnrad neben **About**. Topics nur Kleinbuchstaben, Ziffern, Bindestriche; einzeln eintippen und mit Enter bestätigen.

> **Tipp:** Hast du auf GitHub etwas geändert (z. B. die Lizenz angelegt), hole diese Änderung vor dem Weiterarbeiten auf deinen Rechner: `git pull`. Mehr dazu in Kapitel 9.

9 Weiterarbeiten: lokal und im Web

Ab jetzt existiert dein Projekt an zwei Orten. Die wichtigste Regel: Beide Stände müssen zusammenpassen.

Schritt 9.1: Vor jeder Arbeitssitzung

```
cd /c/GitHub/<PROJEKT>
git fetch
git status
```

Meldet `git status` „Your branch is behind“, liegt online Neueres, etwa aus der Weboberfläche. Dann holst du es ab:

```
git pull
```

Schritt 9.2: Nach getaner Arbeit

```
git add .
git commit -m "<NACHRICHT>"
git push
```

> **Tipp:** Arbeitest du auch über die Weboberfläche oder von einem zweiten Gerät aus, gewöhne dir an: **Vorher pullen, nachher pushen.** Dann laufen die Stände nie auseinander.

Schritt 9.3: Von Private auf Public umstellen

Ist das Projekt reif für die Öffentlichkeit:

- Vorher die gesamte Geschichte prüfen, denn sie wird mit veröffentlicht: `git log --format="%h %ae %s"`. Keine echte Mailadresse, keine Zugangsdaten?
- Auf GitHub: **Settings** → ganz unten **Danger Zone** → **Change visibility** → Public.

10 Einen fertigen Stand als Release veröffentlichen

Ein Release markiert einen Commit dauerhaft als Version. Besucher sehen sofort, welche Fassung die aktuelle ist, und spätere Änderungen überschreiben sie nicht.

Schritt 10.1: Vorbereitung

- Alles committet und hochgeladen? `git status` meldet „up to date“ und „working tree clean“.
- Dokumente auf Entwurfsvermerke prüfen, Versionsnummern abgleichen.

Schritt 10.2: Release anlegen (Weboberfläche)

- Auf der Repo-Startseite rechts unter **Releases** → **Create a new release**.
- **Choose a tag** → z. B. `v1.0` eintippen → **Create new tag on publish**.
- **Release title:** z. B. „Version 1.0“.
- **Release notes:** was enthalten ist und was sich geändert hat.
- Optional: Dateien für Leser ohne Git-Kenntnisse (z. B. eine PDF) unter **Attach binaries** anhängen.
- **Release label** auf „None“ lassen; „Pre-release“ nur für unfertige Stände.
- **Publish release**.

Nach dem Release holst du die neue Markierung auch auf deinen Rechner:

```
git fetch --tags
```

11 Wenn etwas klemmt

Die häufigsten Meldungen und was sie bedeuten:

Meldung	Ursache	Lösung
„detected dubious ownership in repository“	Der Ordner gehört nicht deinem Benutzerkonto, häufig nach Anlegen mit Administratorrechten.	Besitzer des Ordners auf das eigene Konto ändern (Schritt 4.4). Notlösung: `git config --global --add safe.directory C:/GitHub/<PROJEKT>`
„! [rejected] main -> main (fetch first)“	Online liegen Commits, die du lokal nicht hast.	`git pull`, dann erneut `git push`
„GH007: Your push would publish a private email address“	Ein Commit trägt deine echte Mailadresse.	Schritt 4.2 wiederholen. Betrifft es nur den letzten Commit: `git commit --amend --reset-author --no-edit`, dann `git push`

Meldung	Ursache	Lösung
„LF will be replaced by CRLF“	Hinweis zur Angleichung der Zeilenenden. Bei Textdateien harmlos.	Erscheint die Meldung bei einer PDF oder einem Bild: `.gitattributes` prüfen (Schritt 6.3) und die Datei mit `git restore <DATEI>` zurückholen, nicht committen.
„No such file or directory“ bei `cd`	Pfad falsch geschrieben oder Leerzeichen im Namen.	Pfad in Anführungszeichen setzen, Tab-Ergänzung nutzen.
„fatal: not a git repository“	Du bist nicht im Projektordner.	`cd /c/GitHub/<PROJEKT>`
„remote origin already exists“	Die Verbindung zu GitHub ist schon eingerichtet.	Mit `git remote -v` prüfen. Falsche Adresse korrigieren: `git remote set-url origin <ADRESSE>`
Anzeige hängt, unten steht „:“ oder „(END)“	Git zeigt eine lange Ausgabe seitenweise an.	Taste **q** beendet die Anzeige.

Notfall: Zugangsdaten versehentlich committet

Noch nicht hochgeladen und nur im letzten Commit:

```
# Datei aus Git entfernen, aber auf der Festplatte behalten
git rm --cached secrets.py

# In .gitignore eintragen, falls noch nicht geschehen
echo "secrets.py" >> .gitignore
git add .gitignore

# Letzten Commit ohne die Datei neu schreiben
git commit --amend --no-edit
```

Bereits hochgeladen: Betroffenes Passwort **sofort ändern**. Das ist der einzige wirklich sichere Schritt, denn wer die Daten schon abgegriffen hat, behält sie. Die Bereinigung der Versionsgeschichte ist danach ein zweiter, aufwendigerer Schritt, für den du dir Unterstützung holen solltest.

> **Achtung:** Vor jedem Eingriff in die Versionsgeschichte den ganzen Projektordner kopieren, inklusive des versteckten `.git`-Ordners: `cp -r "<PROJEKT>" "_Backup_<PROJEKT>"`

12 Befehlsübersicht

Befehl	Wirkung
`git status`	Zustand: geänderte und neue Dateien, Abstand zu GitHub
`git init`	Ordner zum Repo machen
`git add <DATEI> / git add .`	Datei / alles für den nächsten Commit vormerken
`git commit -m "<NACHRICHT>"`	Vorgemerktes festschreiben
`git diff`	Änderungen zeilengenau anzeigen
`git log --format="%h %an <%ae> %ad %s" --date=short`	Commits mit Autor, Mail, Datum
`git remote add origin <ADRESSE>`	Verbindung zu GitHub herstellen
`git remote -v`	Verbindung anzeigen
`git push -u origin main`	Erster Upload
`git push`	Lokale Commits hochladen
`git fetch`	Online-Stand ansehen, ohne lokal etwas zu ändern
`git pull`	Online-Stand holen und übernehmen
`git restore <DATEI>`	Lokale Änderung verwerfen
`git restore --staged <DATEI>`	Datei aus dem Vormerken nehmen
`git rm --cached <DATEI>`	Datei aus Git entfernen, auf Platte behalten
`git config --global --list`	Einstellungen anzeigen

13 Checkliste

Zum Abhaken für jedes neue Projekt. Die ersten beiden Blöcke erledigst du nur einmal pro Rechner.

Einmalig: GitHub-Konto

☐ „Keep my email addresses private“ aktiviert, Ersatzadresse notiert

☐ „Block command line pushes that expose my email“ aktiviert

Einmalig: Rechner

☐ Git für Windows installiert, `git --version` funktioniert

☐ `user.name` und `user.email` (Ersatzadresse!) gesetzt

☐ `init.defaultBranch main`, `core.autocrlf true`, `pull.rebase false` gesetzt

☐ Arbeitsordner `C:\GitHub` angelegt, außerhalb von OneDrive, Besitzer ist das eigene Konto

Planung

☐ Projektname ohne Leerzeichen und Umlaute festgelegt

☐ Projektart bestimmt, Ordnerstruktur und Lizenz passend gewählt (Schritt 5.2)

☐ Zugangsdaten nur in `secrets.py`, Vorlage `secrets_example.py` mit Platzhaltern

☐ Keine privaten Angaben, keine ZIPs, keine fremden Handbücher im Ordner

☐ README mit Beschreibung, Voraussetzungen, Installation

Lokales Repo

☐ `git init` ausgeführt

☐ `.gitignore` angelegt, `secrets.py` eingetragen

☐ `.gitattributes` angelegt, Binärdateien eingetragen

☐ `git status` geprüft: keine Zugangsdaten unter „Untracked files“

☐ Erster Commit erstellt

☐ `git log` geprüft: Ersatzadresse im Commit

Veröffentlichung

☐ Leeres Repo auf GitHub erstellt, ohne README/Lizenz-Häkchen, zunächst Private

☐ `git remote add origin` ausgeführt, mit `git remote -v` geprüft

☐ `git push -u origin main` erfolgreich, Anmeldung per Browser erledigt

☐ Dateien auf GitHub sichtbar, `secrets.py` fehlt

☐ Lizenz angelegt, Beschreibung und Topics gesetzt

☐ Lokal `git pull` nach Änderungen auf GitHub

Öffentlich machen und Release

☐ Gesamte Geschichte auf Mailadressen und Zugangsdaten geprüft

☐ Sichtbarkeit auf Public umgestellt

☐ Dokumente ohne Entwurfsvermerk, Versionsnummern stimmen

☐ Release mit Tag, Titel und Beschreibung veröffentlicht

☐ Bei Bedarf PDF oder andere Dateien als Download angehängt

Im Alltag

☐ Vor der Arbeit: `git fetch`, `git status`, ggf. `git pull`

☐ Nach der Arbeit: `git add`, `git commit` mit aussagekräftiger Nachricht, `git push`